

Módulo 8 — CRUD no Protheus: AxCadastro e mBrowse

Jornada DEV START · Aula 8 · 28/07 (terça) · Programa START — TOTVS Paulista
Material entregue ao final da Aula 8. Guarde para consultar sempre que precisar.



Onde você está na jornada

```
Fundamentos (Harbour)          Protheus (ADVPL)
1 → 2 → 3 → 4 → 5 → 6  ⇒  7 → [8] → 9 → 10
                        ▲
                    você está aqui
```

Ontem você entendeu o **terreno** (arquitetura + dicionário de dados). Hoje é dia de **construir**: você vai escrever seu **primeiro programa ADVPL**, criar uma **tabela própria** e montar uma **tela de cadastro completa** — com incluir, alterar, excluir e consultar.

A aula tem **três tempos**: 1. **Primeiro programa** — `USER FUNCTION` + `MsgInfo` + compilar (F9) + rodar pelo SmartClient. 2. **AxCadastro** — o jeito **rápido**: CRUD funcional com pouquíssimas linhas. 3. **mBrowse** — o jeito **profissional**: mais controle, legendas coloridas, filtros, F3 e gatilhos.

No fim, você vai saber **quando usar cada um**. 🎯



Penúltima aula do curso. O programa START encerra em **29/07**. A entrega dos exercícios deste módulo vai até **03/08**. Os itens de `mBrowse`, legendas e filtro (mais avançados) ficam como **material bônus de autoestudo** — não bloqueiam sua entrega.



Roteiro da aula (2h)

Bloco	Tempo	O quê
Retomada	~10 min	Recap do dicionário (SX2/SX3, Real vs Virtual) + <code>USER FUNCTION</code>
Primeiro programa	~10 min	<code>USER FUNCTION</code> + <code>MsgInfo</code> + compilar (F9) + rodar no SmartClient
Conteúdo	~50 min	Tabela ZA1 (Pets), funções fundamentais, <code>AxCadastro</code> → <code>mBrowse</code>
Mão na massa	~40 min	Montar o CRUD de Pets: primeiro <code>AxCadastro</code> , depois <code>mBrowse</code>
Dúvidas / networking	~10 min	Perguntas abertas e integração da turma



Objetivos do módulo

Ao final desta aula você será capaz de: - Escrever, compilar (F9) e executar seu **primeiro programa ADVPL** pelo SmartClient (**Miscelânea > Execução > Programa**) - Usar `#include "protheus.ch"` e `USER FUNCTION` com fluência - Criar e configurar uma **tabela customizada** (ZA1) no dicionário de dados - Dominar as funções fundamentais: `xFilial`, `POSICIONE`, `GetSXEnum`, `ExistChav`, `ExistCpo` - Montar um CRUD completo com `AxCadastro` (rápido) e personalizar botões via `aRotina` - Evoluir para `mBrowse` (profissional) com **legendas coloridas**, **filtros**, **SXB (F3)** e **gatilhos (SX7)** - Decidir **quando usar** `AxCadastro` e **quando usar** `mBrowse`

8.1 Retomada: de Harbour para ADVPL

Você já sabe programar em Harbour. Ao entrar no ADVPL, três coisas mudam:

1. Ponto de entrada: `USER FUNCTION` no lugar de `FUNCTION Main()`.

```
// Harbour
FUNCTION Main()
    QOut("Hello!")
RETURN NIL

// ADVPL
#include "protheus.ch"
USER FUNCTION MEUPROGRAMA()
    MsgInfo("Hello!", "Bem-vindo")
RETURN NIL
```

2. Interface gráfica: funções de mensagem e formulário.

```
MsgInfo("Mensagem informativa", "Título")
MsgAlert("Alerta importante!", "Atenção")
MsgStop("Erro crítico!", "Erro")
IF MsgYesNo("Confirma a exclusão?", "Confirmar")
    // usuário clicou em Sim
ENDIF
```

3. Nomenclatura de arquivos e funções: - Arquivos: **.PRW** (Protheus) ou **.PRG** . - Nome da **USER FUNCTION** : convenção pelo prefixo do módulo/cliente. - **SIGACOM** → **STCOM001.PRW** - Módulo customizado do curso → prefixo **ST** : **STTIP001.PRW**

 **Recap do dicionário (Módulo 7):** SX2 = tabelas, SX3 = campos, SX7 = gatilhos, SIX = índices, SXB = consultas F3. E o mais importante para hoje: **Real** (gravado) vs **Virtual** (calculado).

8.2 Seu primeiro programa ADVPL

Isso ficou pendente da aula anterior — então vamos fazer agora, antes de qualquer coisa. É o “Hello, World!” do Protheus: pouco código, mas o ritual completo (escrever → compilar → rodar) é o mesmo que você vai repetir centenas de vezes na carreira.

```
// =====
// STTIP000.PRW
// Primeiro programa ADVPL
// =====

#include "protheus.ch"

USER FUNCTION STTIP000()
    MsgInfo("Olá, Protheus!", "Bem-vindo")
RETURN NIL
```

Passo a passo: 1. Crie o arquivo `STTIP000.PRW` com o código acima. 2. **Compile com F9** no seu ambiente de desenvolvimento (DevStudio/TDS). Tem que aparecer “*Compilação concluída*” sem erro. 3. No **SmartClient**: menu **Miscelânea > Execução > Programa** → digite `STTIP000` → **OK**. 4. Deve aparecer a caixa `MsgInfo` com “Olá, Protheus!”.

✅ **Resultado esperado:** a caixa de mensagem na tela. Se aparecer, você acabou de rodar seu primeiro programa ADVPL de ponta a ponta — é exatamente esse ciclo (escrever, compilar, executar) que vamos repetir o resto da aula com a `AxCadastro` e o `mBrowse`.

8.3 Nossa tabela de exemplo: ZA1 — Pets do cliente

🐾 **Resolvendo a pendência de ontem:** na aula de 27/07, ao vivo no Configurador, a turma criou a tabela **ZA1** para cadastrar **pets** (nome, raça, data de nascimento) — mas ficou faltando a **amarração com o cliente** (quem é o dono?). É exatamente isso que fechamos agora.

O Protheus reserva o prefixo **Z** para **tabelas do usuário** (as suas). Vamos completar a tabela **ZA1 — Pets do cliente**, que registra os pets de cada cliente:

Descrição	Campo	Tipo	Tam	Dec	Contexto
Filial	ZA1_FILIAL	C	2	0	Real
Código	ZA1_COD	C	6	0	Real
Cliente (dono)	ZA1_CLIENT	C	6	0	Real
Loja do cliente	ZA1_LOJA	C	2	0	Real
Nome do cliente	ZA1_NOMCLI	C	40	0	Virtual
Nome do pet	ZA1_NOME	C	30	0	Real
Raça	ZA1_RACA	C	20	0	Real
Nascimento	ZA1_DTNASC	D	8	0	Real
Observação	ZA1_OBS	C	60	0	Real

Repare no **ZA1_NOMCLI** : ele é **Virtual** — não é gravado no banco. É **calculado** a partir do dono do pet (**ZA1_CLIENT** + **ZA1_LOJA**), buscando o nome na SA1. Aquele conceito do Módulo 7, na prática! E é essa dupla (**ZA1_CLIENT** + **ZA1_LOJA**) que amarra o pet ao cliente — **a ZA1 estava solta, sem dono; agora o pet pertence a um cliente.**

Passos no Configurador (SIGACFG)

1. Cadastrar a tabela (SX2): - Menu: **Dicionário > Banco de Dados > Dicionário de Dados** - Botão **Incluir** - Preencha: Prefixo = **ZA1** , Nome = **Pets** , Modo = **C** (Compartilhado)

Se a turma já criou a ZA1 ao vivo na aula anterior, o trabalho de hoje é **completar/revisar** os campos abaixo — em especial o **ZA1_CLIENT** , **ZA1_LOJA** e o **ZA1_NOMCLI** virtual, que fazem a amarração com o cliente.

2. Adicionar os campos (SX3):

Campo **ZA1_COD** — o código sequencial: - Tipo: **C** , Tamanho: **6** , Contexto: **Real** - Browse: **Sim** (aparece na listagem)

Campo **ZA1_CLIENT** — o dono do pet, com lookup para a SA1: - Tipo: **C** , Tamanho: **6** - **F3:** **SA1** (ou uma consulta padrão que você criar) - **Validação:** **ExistCpo("SA1", xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, 1)**

Campo **ZA1_NOMCLI** — campo **Virtual** (calculado): - Contexto: **Virtual** (não vai para o banco) - **Relação:** **POSICIONE("SA1",1,xFilial("SA1")+M->ZA1_CLIENT+M->ZA1_LOJA,"A1_NOME")**

 Configurador: campos da tabela ZA1 no SX3

3. Criar índices (SIX): - Índice 1: `ZA1_FILIAL + ZA1_COD` (chave primária) - Índice 2: `ZA1_FILIAL + ZA1_CLIENT + ZA1_LOJA` (busca por cliente — os pets de um dono)

4. Criar consulta padrão ZA1 (SXB): Permite que outros campos usem a ZA1 como lookup (veremos o SXB no item 8.11).

8.4 Funções fundamentais do ADVPL

Estas cinco funções aparecem em praticamente **toda** rotina de CRUD. Domine-as.

`xFilial(cAlias)`

Retorna o código da filial para filtrar registros da tabela. **Essencial** em qualquer acesso a banco.

```
LOCAL cChave := xFilial("SA1") + cCodCli + cLoja
// Ex.: "01" + "000001" + "01" → "010000011"
```

Por que é necessário? Em tabela **compartilhada**, o `FILIAL` é " " (espaços). Em **exclusiva**, é o código da filial. `xFilial()` devolve o valor certo — sem chute.

`POSICIONE(cAlias, nOrdem, cChave, cCampo)`

Busca um valor em outra tabela **sem mexer** na posição atual do ponteiro. Muito usado em campos virtuais.

```
// Nome do cliente (dono do pet) na SA1
cNomeCliente := POSICIONE("SA1", 1, xFilial("SA1") + cCodCli + cLoja, "A1_NOME")

// Descrição de um produto na SB1
cDescProd := POSICIONE("SB1", 1, xFilial("SB1") + cCodProd, "B1_DESC")
```

Parâmetros: `cAlias` (tabela), `nOrdem` (índice), `cChave` (chave montada conforme o índice), `cCampo` (campo a retornar).

`GetSXENum(cAlias, cCampo)`

Gera o **próximo número sequencial** para um campo código (usa a tabela SX8 por baixo).

```
LOCAL cCodigo := GetSXENum("ZA1", "ZA1_COD")  
// Resultado: "000001", "000002", ...
```

ExistChav(cAlias, cChave, nOrdem)

Verifica se uma chave **já existe** na tabela — usado para garantir **unicidade**.

```
IF ExistChav("ZA1")  
    MsgAlert("Código já cadastrado!", "Atenção")  
    RETURN .F.  
ENDIF
```

ExistCpo(cAlias, cChave, nOrdem)

Verifica se uma chave existe em **outra** tabela — usado para validar **referências**.

```
// Valida se o cliente (dono do pet) existe na SA1  
IF !ExistCpo("SA1", xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, 1)  
    MsgAlert("Cliente não cadastrado!", "Atenção")  
    RETURN .F.  
ENDIF
```

8.5 AxCadastro — CRUD automático (o jeito rápido)

AxCadastro cria automaticamente uma **tela de CRUD completa** para uma tabela do dicionário. É o caminho mais rápido para ter um cadastro funcionando.

Sintaxe

```
AxCadastro(cAlias, cTitulo, cCpo, cOrdem, cBusca, cDelta, cDeltaMemo, lMemoria,  
lSemFil)
```

Parâmetros principais: - **cAlias** — tabela (ex: "ZA1") - **cTitulo** — título da tela (ex: "Pets") - **cCpo** — campos do browse separados por ; (vazio = usa o SX3) - **cOrdem** — número da ordem (índice)

Exemplo completo: rotina de Pets

```
// =====  
// STTIP001.PRW  
// Cadastro de Pets (AxCadastro)  
// =====  
  
#include "protheus.ch"  
  
USER FUNCTION STTIP001()  
  
    PRIVATE cCadastro := "Pets"  
  
    dbSelectArea("ZA1")  
    dbSetOrder(1)  
  
    AxCadastro("ZA1", "Pets", , "1", , , , .F.)  
  
RETURN NIL
```

Com **essas poucas linhas**, o Protheus monta automaticamente: -  **Listagem (Browse)** dos registros -  Botão **Incluir** (abre o formulário de cadastro) -  Botão **Alterar** -  Botão **Excluir** -  Botão **Visualizar** -  **Pesquisa** por índice -  **Validações** configuradas no dicionário



Tela do AxCadastro: browse com os botões de CRUD



Formulário de inclusão gerado pelo AxCadastro

8.6 aRotina — personalizando os botões

Você controla quais botões aparecem (e adiciona os seus) com a variável **aRotina** :


```

USER FUNCTION STTIP001()

PRIVATE cCadastro := "Pets"
PRIVATE aRotina := {;
    {"Pesquisar", "AxPesqui", 0, 1},;
    {"Visualizar", "AxVisual", 0, 2},;
    {"Incluir", "AxInclui", 0, 3},;
    {"Alterar", "AxAltera", 0, 4},;
    {"Excluir", "AxDeleta", 0, 5};
}

dbSelectArea("ZA1")
dbSetOrder(1)

AxCadastro("ZA1", "Pets")

RETURN NIL

```

Cada linha do **aRotina** tem **4 elementos**: 1. Texto do botão 2. Função a chamar 3. Parâmetro (geralmente **0**) 4. Tipo da operação: **1**=Pesq, **2**=Vis, **3**=Inc, **4**=Alt, **5**=Del, **6**=customizado

 Barra de botões do aRotina no cadastro

Adicionando um botão customizado

```

PRIVATE aRotina := {;
    {"Pesquisar", "AxPesqui", 0, 1},;
    {"Visualizar", "AxVisual", 0, 2},;
    {"Incluir", "AxInclui", 0, 3},;
    {"Alterar", "AxAltera", 0, 4},;
    {"Excluir", "AxDeleta", 0, 5},;
    {"Relatório", "U_STTIP001REL", 0, 6}; // botão customizado!
}

```

8.7 Validações no código

Quando você usa **AxInclui** ou **AxAltera**, as validações do **SX3** rodam **automaticamente**. Para regras mais complexas, crie uma **USER FUNCTION de validação** e aponte no **X3_VALID**:

```
// No SX3 de ZA1_CLIENT: X3_VALID = "U_VALCLI001()"

USER FUNCTION VALCLI001()
  // M-> acessa o conteúdo do campo atual no formulário
  IF !ExistCpo("SA1", xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, 1)
    MsgAlert("Cliente não cadastrado!", "Atenção")
    RETURN .F.    // .F. = inválido (campo continua em edição)
  ENDIF
  RETURN .T.    // .T. = válido (campo aceito)
```

🧠 **M->** é o “aqui e agora” do formulário: acessa o valor que o usuário **está digitando**, antes de gravar. Diferente de **ZA1->ZA1_CLIENT**, que lê o valor **já gravado** no registro.

Rotina completa com filtro inicial

```
USER FUNCTION STTIP001()

PRIVATE cCadastro := "Pets"
PRIVATE aRotina := {
  {"Pesquisar", "AxPesqui", 0, 1},;
  {"Visualizar", "AxVisual", 0, 2},;
  {"Incluir", "AxInclui", 0, 3},;
  {"Alterar", "AxAlterar", 0, 4},;
  {"Excluir", "AxDeleta", 0, 5};
}

dbSelectArea("ZA1")
dbSetOrder(1)
dbSeek(xFilial("ZA1"))    // posiciona na filial atual

AxCadastro("ZA1", "Pets", , "1", , , , .F.)

RETURN NIL
```

Adicionando a rotina ao menu

Para o usuário acessar pelo menu do Protheus: 1. Abra o **Configurador** → **Menu do Sistema**
 2. Selecione o módulo (ex: **SIGACOM**) 3. Adicione o item: Texto = **"Pets"**, Função = **STTIP001**, Módulo = **SIGACOM**



Configurador: adicionando a rotina ao Menu do Sistema

8.8 De AxCadastro para mBrowse: quando usar cada um

Você já tem um CRUD funcionando com pouquíssimo código. Então **por que existe o mBrowse** ? Porque no mundo real você vai precisar de mais **controle**.

	AxCadastro	mBrowse
Código	Mínimo	Mais detalhado
Controle da interface	Baixo	Alto
Legendas coloridas	✗ Não	✓ Sim
Filtros dinâmicos	Limitado	✓ Completo
Colunas customizadas	✗ Não	✓ Sim
Uso típico	Protótipos, tabelas simples	Rotinas de produção

 **A regra prática:** use **AxCadastro** para **aprender e prototipar** — quando você só precisa do CRUD no ar rápido. Use **mBrowse** para o **sistema real** — quando precisa de legendas, filtros, colunas sob medida e comportamento profissional.

O bom é que a base é a mesma: **cCadastro**, **aRotina** e o dicionário continuam valendo. Você só **troca a função de abertura** — e ganha superpoderes.

 **Material bônus:** como esta é a penúltima aula do curso, o conteúdo de **mBrowse** a partir daqui (legendas, filtros, F3) fica registrado como **material de autoestudo**. Vale a pena ler com calma, mas não é obrigatório para a entrega — o essencial do módulo é a Seção 8.1 a 8.7 (primeiro programa + **AxCadastro** com a ZA1 amarrada à SA1).

8.9 mBrowse — o jeito profissional (*bônus/autoestudo*)

Sintaxe

```
mBrowse(nLin1, nCol1, nLin2, nCol2, cAlias, aFixe, cCpo, nPar, ;  
        cCorFun, nClickDef, aColors, cTopFun, cBotFun, nPar14, ;  
        bInitBloc, lNoMnuFilter, lSeeAll, lChgAll, ;  
        cExprFilTop, nInterval)
```

São muitos parâmetros, mas na prática os que você mais usa são: - `nLin1`, `nCol1`, `nLin2`, `nCol2` — posição na tela (padrão: `1, 1, 22, 75`) - `cAlias` — tabela (ex: `"ZA1"`) - `aFixe` — colunas fixas (geralmente `NIL`) - `aColors` — **regras de cores** por linha (as legendas!) - `cTopFun` / `cBotFun` — filtro de início/fim - `cExprFilTop` — filtro adicional (WHERE)

Estrutura mínima

```
// =====  
// STTIP002.PRW  
// Cadastro de Pets (mBrowse)  
// =====  
  
#include "protheus.ch"  
  
USER FUNCTION STTIP002()  
  
    LOCAL cFiltro := ""  
  
    PRIVATE cCadastro := "Pets"  
    PRIVATE aRotina := {  
        {"Pesquisar", "AxPesqui", 0, 1},;  
        {"Visualizar", "AxVisual", 0, 2},;  
        {"Incluir", "AxInclui", 0, 3},;  
        {"Alterar", "AxAltera", 0, 4},;  
        {"Excluir", "AxDeleta", 0, 5};  
    }  
  
    dbSelectArea("ZA1")  
    dbSetOrder(1)  
  
    mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , , cFiltro)  
  
    RETURN NIL
```

 Tela do mBrowse com a listagem de pets

Note que o **aRotina** continua igual ao do `AxCadastro` — as funções `AxInclui` / `AxAltera` /etc. são reaproveitadas. O que muda é a **função de abertura** (`mBrowse` no lugar de `AxCadastro`).

8.10 Legendas coloridas (aColors) (bônus/autoestudo)

Uma das funcionalidades mais poderosas do **mBrowse** : **pintar as linhas** conforme regras de negócio.

Por que usar legendas?

Imagine uma lista de pets. Você quer que o usuário **veja na hora** quais precisam de atenção: -

● Vermelho: pet com aniversário atrasado há mais de 30 dias (data de nascimento antiga demais para o filtro que você quiser aplicar) - ● Amarelo: pet que faz aniversário hoje - ● Verde: os demais

Configurando **aColors**

```
LOCAL aColors := {;  
    {"ZA1->ZA1_DTNASC < dDataBase - 30", "BR_RED"},;    // vermelho  
    {"Mes(ZA1->ZA1_DTNASC) == Mes(dDataBase) .And. Dia(ZA1->ZA1_DTNASC) ==  
    Dia(dDataBase)", "BR_YELLOW"},; // amarelo: aniversário hoje  
    {"T.", "BR_GREEN"},; // verde (padrão)  
}  
  
mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , , cFiltro)
```

⚠ **A ordem importa!** As regras são avaliadas **de cima para baixo** — a **primeira** que for verdadeira define a cor da linha. Por isso a regra **"T."** (sempre verdadeira) fica **por último**, funcionando como cor padrão.

Cores disponíveis

Constante	Cor	Constante	Cor
BR_RED	Vermelho	BR_CYAN	Ciano
BR_GREEN	Verde	BR_MAGENTA	Magenta
BR_YELLOW	Amarelo	BR_BLACK	Preto
BR_BLUE	Azul	BR_WHITE	Branco

 mBrowse com legendas coloridas por regra de negócio

8.11 Filtros dinâmicos (*bônus/autoestudo*)

O `mBrowse` deixa o usuário **filtrar os dados na própria tela**. Para habilitar o menu de filtro, passe `lNoMnuFilter = .F.:`

```
mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , .F., , , cFiltro)
```

Você também pode passar um **filtro pré-definido** por código:

```
// Mostrar apenas pets de um cliente específico
LOCAL cFiltro := "ZA1_CLIENT == '000001'"

mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , cFiltro)
```

 mBrowse: menu de filtro dinâmico aberto

8.12 Consultas padrão (SXB) — o lookup do F3 (*bônus/autoestudo*)

Quando o usuário pressiona **F3** em um campo, abre uma **tela de consulta**. Isso é configurado no **SXB** e depois apontado no campo pelo `X3_F3`. Vamos usar essa consulta exatamente no campo `ZA1_CLIENT`, para o usuário buscar o cliente (dono do pet) sem digitar o código de cabeça.

Estrutura do SXB

Campo	Descrição
BIX_TIPO	Tipo: 1=Busca, 2=Gatilho, 3=Consulta, 4=Browse
BIX_QUERY	Consulta a que pertence
BIX_CAMPO	Campo
BIX_CONTEUDO	Conteúdo

Consulta padrão para o F3 do `ZA1_CLIENT` (usando a SA1)

Como o `ZA1_CLIENT` busca um **cliente**, a consulta padrão de F3 desse campo é a `SA1010` (consulta padrão de clientes que já existe no Protheus) — não é preciso criar uma nova. No SX3 do campo: `X3_F3 = "SA1010"`.

Se quiser, você também pode criar uma consulta padrão própria da **ZA1** (para outros campos que um dia queiram “buscar um pet”), seguindo o mesmo padrão:

Tipo 1 — Cabeçalho (alias):

```
BIX_TIPO = "1"    BIX_QUERY = "ZA1001"    BIX_CAMPO = "ALIAS"    BIX_CONTEUDO = "ZA1"
```

Tipo 5 — Índice de busca:

```
BIX_TIPO = "5"    BIX_CAMPO = "ORDENS"    BIX_CONTEUDO = "1"    // usa o índice 1 da ZA1
```

Tipo 3 — Colunas do browse:

```
Seq 01: ZA1_COD,      título "Código"  
Seq 02: ZA1_NOME,     título "Nome do pet"  
Seq 03: ZA1_RACA,     título "Raça"
```

Tipo 4 — Retorno (o que preenche no campo de origem):

```
BIX_CAMPO = "ZA1_COD"
```

 Consulta padrão (F3) aberta a partir de um campo

8.13 Gatilhos (SX7) na prática

Gatilhos executam **automaticamente** quando um campo é preenchido. Vamos fazer o **ZA1_CLIENT** preencher o **ZA1_NOMCLI** sozinho — **esta é a peça que fecha a pendência da aula de 27/07**: a ZA1 deixa de ser uma tabela solta e passa a “saber” quem é o dono do pet, exibindo o nome do cliente automaticamente.

No **Configurador > Dicionário > Gatilhos > Incluir**:

Campo SX7	Valor
Seqüência	001
Campo	ZA1_CLIENT
Ordem	1
Campo Destino	ZA1_NOMCLI
Condição	vazio ou INCLUI .OR. ALTERA
Tipo	P = Primário
Regra	POSICIONE("SA1",1,xFilial("SA1")+M->ZA1_CLIENT+M->ZA1_LOJA,"A1_NOME")
Fase	1 = Não positiva

O que acontece: ao sair do campo **ZA1_CLIENT**, o Protheus executa a regra e preenche o **ZA1_NOMCLI** **sem o usuário digitar**. Crie também o gatilho para **ZA1_LOJA** (atualiza o nome quando a loja muda).

 Configurador: configuração de gatilho (SX7) ZA1_CLIENT → ZA1_NOMCLI

 **Gatilho x campo Virtual:** os dois preenchem o nome do cliente, mas de formas diferentes. O **campo Virtual** calcula na **exibição** (não grava). O **gatilho** preenche um campo **Real** no momento da digitação (grava). Escolha conforme a necessidade de ter o valor **persistido** ou não.

Bônus: a brincadeira do CEP

Gatilho não serve só para “trazer o nome de outra tabela” — ele também pode **chamar uma função sua** e fazer contas. Fizemos essa brincadeira com o **CEP** do cadastro de Clientes: um gatilho no campo **A1_CEP** que preenche **bairro, município e UF** automaticamente, chamando a **U_STCEP()**.

A ideia: você digita o CEP na tela de Clientes, sai do campo (Tab), e o Protheus preenche os três campos de endereço sozinho — três gatilhos no mesmo campo **A1_CEP**, cada um com um contra-domínio diferente (**A1_BAIRRO**, **A1_MUN**, **A1_EST**) e a mesma função por trás, variando o segundo parâmetro:

```
U_STCEP(M->A1_CEP, "BAIRRO")
U_STCEP(M->A1_CEP, "CIDADE")
U_STCEP(M->A1_CEP, "UF")
```


O material completo (passo a passo de compilação, amarração dos 3 gatilhos no SX7, teste e exercício escrito) está em `cep/GATILHO-CEP.md`, e o código-fonte pronto em `cep/stcep.prw` — ambos nesta mesma pasta do módulo. Vale a pena fazer com calma: é o mesmo mecanismo de gatilho da ZA1, só que chamando uma função sua em vez de um `POSICIONE`.

8.14 Rotina profissional completa (*bônus/autoestudo*)

Juntando tudo — `mBrowse` + legendas + filtro:

```
#include "protheus.ch"

USER FUNCTION STTIP002()

    LOCAL cFiltro := ""
    LOCAL aColors

    PRIVATE cCadastro := "Pets"
    PRIVATE aRotina := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAlterar", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5};
    }

    // Legendas de cores:
    // Vermelho: nascido há mais de 30 dias contados a partir de hoje (exemplo didático)
    // Amarelo: aniversário hoje
    // Verde: os demais
    aColors := {;
        {"ZA1->ZA1_DTNASC < dDataBase - 30", "BR_RED"},;
        {"ZA1->ZA1_COD == GetSXEnum('ZA1','ZA1_COD')", "BR_YELLOW"},;
        {"T.", "BR_GREEN"};
    }

    dbSelectArea("ZA1")
    dbSetOrder(1)
    dbSeek(xFilial("ZA1"))

    mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , .F., , , cFiltro)

    RETURN NIL
```

Bônus: colunas personalizadas

```
PRIVATE aColunas := {;  
    {"ZA1_COD",    "Código",    6},;  
    {"ZA1_NOMCLI", "Cliente",   30},;  
    {"ZA1_NOME",   "Pet",       30},;  
    {"ZA1_RACA",   "Raça",      20};  
}  
  
mBrowse(1, 1, 22, 75, "ZA1", aColunas, , , , , aColors, , , , , .F., , , cFiltro)
```

👉 Mão na massa (em aula)

⚠️ **Este bloco depende do ambiente Protheus** (SmartClient + AppServer + Configurador). Sem ele, você ainda pode **escrever e comentar o código** (a lógica ADVPL não muda) e **descrever os passos** de dicionário. A execução das telas exige o ambiente.

Vamos construir o CRUD de Pets em **dois tempos**, ao vivo:

Tempo 0 — o primeiro programa: 0. Escrever `STTIP000.PRW` com `MsgInfo`, compilar (F9) e rodar via **Miscelânea > Execução > Programa**.

Tempo 1 — o jeito rápido (`AxCadastro`): 1. Revisar/completar a tabela **ZA1** no Configurador (SX2, campos no SX3, índices no SIX) — fechando a amarração `ZA1_CLIENT` + `ZA1_LOJA` com a SA1. 2. Configurar o `ZA1_NOMCLI` como **Virtual** com `POSICIONE`. 3. Escrever `STTIP001.PRW` com `AxCadastro`, compilar (F9) e rodar no SmartClient. 4. Incluir 2–3 pets de teste, amarrados a clientes já existentes.

Tempo 2 — o jeito profissional (`mBrowse`) — bônus, se der tempo: 5. Criar `STTIP002.PRW` trocando `AxCadastro` por `mBrowse`. 6. Adicionar **legendas coloridas** (`aColors`). 7. Configurar o **gatilho SX7** `ZA1_CLIENT → ZA1_NOMCLI` (e, se sobrar tempo, a brincadeira do CEP).

🎯 Meta: sair da aula com o **CRUD de Pets amarrado ao cliente** funcionando — a pendência de 27/07 resolvida — e o primeiro programa ADVPL rodando do zero.

📌 Referência rápida do Módulo 8

Funções fundamentais:

```

xFilial("SA1") // filial atual da tabela
POSICIONE("SA1",1,xFilial("SA1")+cCod,"A1_NOME") // busca valor em outra tabela
GetSXEnum("ZA1","ZA1_COD") // próximo sequencial
ExistChav("ZA1") // chave já existe aqui?
ExistCpo("SA1", xFilial("SA1")+cCod, 1) // existe em outra tabela?

```

AxCadastro (rápido):

```

PRIVATE cCadastro := "Pets"
dbSelectArea("ZA1") ; dbSetOrder(1)
AxCadastro("ZA1","Pets", , "1", , , , .F.)

```

mBrowse (profissional, bônus):

```

mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , .F., , , cFiltro)
// aColors avaliado de cima p/ baixo; ".T." = cor padrão (por último)

```

aRotina: {"Texto","Funcao",0,Tipo} — 1=Pesq 2=Vis 3=Inc 4=Alt 5=Del 6=Custom

Quando usar: **AxCadastro** = protótipo/simples · **mBrowse** = produção (legendas, filtros, colunas).



Exercícios

Os exercícios desta aula estão em **exercicios.md** (nesta mesma pasta). **Prazo:** até **03/08** (a entrega final do curso).

O essencial é **primeiro programa + ZA1 + AxCadastro**, amarrada à SA1. Os exercícios de **mBrowse**, legendas coloridas e filtro dinâmico são **material bônus de autoestudo** — reforçam o aprendizado, mas não são obrigatórios para concluir o curso. O arquivo indica cada tipo.



Para a próxima aula

Módulo 9 — Projeto CRUD + tratamento de erros. Você vai **juntar tudo** num projeto maior: **duas tabelas relacionadas**, uma **biblioteca de funções comuns**, itens de **menu** e o controle de erros com **BEGIN SEQUENCE ... END SEQUENCE**. É a aula que transforma “rotinas soltas” em **aplicação**.

🐛 Antes da Aula 9: deixe seu CRUD de Pets (AxCadastro e, se fez o bônus, mBrowse) rodando e versionado no GitHub. Travou em algo do dicionário? Poste no Discord [#duvidas-parte2-protheus](#) .